

SISTEM AUTOMASI BATCH-RENAMING FILE DENGAN POLA SEKUENSIAL DAN METODE SORTING VARIATIF

Pemrograman Web Lanjut (PWL)
Teknologi Informasi
Institut Bisnis dan Informatika (IBI) Kesatuan Bogor

Disusun Oleh:

Rhainy Aulia
232310034

1. Project Thesis & Hypothesis Framework

1.1 Problem Statement

Dalam aktivitas ekosistem digital sehari-hari, pengguna seringkali menumpuk aset data dalam jumlah besar dengan format penamaan yang tidak konsisten (misalnya campuran karakter acak, spasi berlebih, atau penanda berkas manual seperti `_final_fix`).

Proses penataan ulang (reorganisasi) secara manual menimbulkan beberapa kendala kritical:

- Membutuhkan waktu eksekusi yang lama (latensi tinggi).
- Kerentanan tinggi terhadap faktor *human error*.
- Risiko inkonsistensi penamaan yang mempersulit pencarian data secara kronologis maupun kategoris.

1.2 Core Hypothesis

Jika *Multi-Criteria Sorting Algorithm* dan *Sequential Renaming Algorithm* diintegrasikan secara sistematis ke dalam utilitas *desktop batch-processing* yang terisolasi, maka:

- Pengelolaan dan tata kelola direktori file menjadi jauh lebih terstruktur.
- Total waktu (latensi) pemrosesan file berkurang secara signifikan dibandingkan metode manual.
- Konsistensi format penamaan massal (*dataset*) meningkat.
- Efisiensi pencarian file berbasis indeks struktural menjadi jauh lebih optimal.

2. Core Algorithmic Engineering

Arsitektur *backend* aplikasi (*core/renaming.py*) bertumpu pada eksekusi paralel dari dua mesin algoritma utama:

2.1 Multi-Criteria Sorting Engine

Berbeda dengan fungsi penyaringan tunggal biasa, mesin ini melakukan pemetaan berbasis *multi-tiered tuple keys*. Algoritma ini mengompilasi urutan file secara hierarkis berdasarkan kombinasi prioritas atribut yang ditentukan pengguna secara dinamis (seperti kombinasi urutan *File Extension*, *Creation Date*, dan *File Size*).

Implementasi Key-Mapping Algorithm (Python):

```
Python
def sort_files_by_multiple_criteria(file_objects,
criteria_priority_list):
    """
        Menghasilkan susunan matriks evaluasi (tuple) berdasarkan
        prioritas
        input pengguna untuk mengurutkan file secara bertingkat.
    """
    def sorting_key_builder(file):
        keys = []
        for criteria in criteria_priority_list:
            if criteria == "Name":
                keys.append(file.name.lower())
            elif criteria == "Size":
                keys.append(file.stat().st_size)
            elif criteria == "Date":
                keys.append(file.stat().st_mtime)
            elif criteria == "Extension":
                keys.append(file.suffix.lower())
        return tuple(keys)

    return sorted(file_objects, key=sorting_key_builder)
```

2.2 Sequential Renaming Engine

Mesin penamaan sekuensial mengeksekusi skema pemformatan string secara massal mengikuti hasil urutan dari *Sorting Engine*. Algoritma ini mengamankan integritas ekstensi file asli, menangani pembersihan simbol, dan menyuntikkan digit penomoran terurut

(*padded serialized numbering*) untuk mencegah terjadinya tumpang tindih nama file (*collision*).

3. Interactive State & UI Feedback Management

Untuk menjaga keamanan sistem (*system safety safety gates*) dan mencegah *user input error*, lapisan antarmuka (`ui/components/renaming_panel.py`) menerapkan arsitektur penguncian komponen visual secara kontekstual:

3.1 Contextual Component Disabling

Setiap kali sebuah opsi pemformatan dipilih, kolom input data atau selektor parameter yang tidak relevan dengan mode aktif tersebut akan secara otomatis dipaksa masuk ke status "disabled" atau "readonly".

3.2 Visual State Styling

Untuk memberikan konfirmasi visual instan, komponen yang terkunci akan melepaskan status fokus aktifnya dan mengubah warna latar belakangnya menjadi abu-abu gelap pekat (#2A2A2A). Hal ini menutup celah bagi *user* untuk memberikan input keyboard yang tidak valid.

Widget State	Foreground Color (fg_color)	Behavioral Action / Impact
Active (normal)	#343638 (Default Dark)	Kolom terbuka penuh dan dapat diedit secara interaktif oleh pengguna.
Locked (disabled)	#2A2A2A (Dark Gray)	Input diabaikan secara absolut oleh sistem; elemen visual redup untuk mencegah kekeliruan.

4. Software Development Life Cycle (SDLC) Methodology

Proses rekayasa perangkat lunak sistem ini mengadopsi metodologi terstruktur yang dibagi ke dalam 5 tahapan utama:

1. **Requirement Analysis:** Mengidentifikasi batasan operasional untuk penanganan multi-kriteria, aturan penomoran sekuensial, visualisasi *live preview*, serta arsitektur log pembatalan tindakan (*undo/rollback storage*).
2. **System Design:** Merancang jalur pipa data (*data pipelines*), pemisahan modular antara logika pemrosesan berkas (*core backend*) dan komponen visual (*ui panels*), serta pemetaan skema file log mutasi lokal (*history.json*).
3. **Development:** Implementasi kode modular Python menggunakan *library* pendukung utama seperti Pathlib untuk manipulasi direktori, CustomTkinter untuk UI, dan JSON untuk manajemen penyimpanan data transaksi lokal.
4. **Testing Run:** Melakukan pengujian fungsionalitas terhadap stabilitas pengurutan bertingkat (*multi-criteria*), validasi pencegahan nama kembar (*conflict detection*), dan efisiensi eksekusi *rollback state*.
5. **Evaluation Phase:** Menilai efisiensi operasional sistem waktu nyata terhadap target performa, dan mengoptimalkan fungsi I/O pada sistem operasi.

5. Standalone Deployment & Production Packaging

Aplikasi ini didistribusikan dalam bentuk *single portable executable* (.exe) menggunakan modul PyInstaller untuk memastikan portabilitas tinggi lintas perangkat Windows tanpa dependensi luar.

5.1 Build Compilation Directive

Proses kompilasi akhir menjadi berkas biner mandiri dieksekusi melalui terminal dengan perintah operasional sebagai berikut:

```
Bash
pyinstaller --onefile --noconsole --icon="app_icon.ico"
--add-data "app_icon.ico;." --name="BatchRenamer" main.py
```

Perintah ini membungkus seluruh dependensi data ke dalam metadata berkas biner, dan memproduksi file .exe final di dalam direktori dist/.